A HoneyBee-Inspired Metaheuristic Multi-Agent Cyber Defense Framework for Adaptive Deception and Autonomous Response

Mithil K Gowda[1,*], GR Dikshith[1], Shaista Tarannum[1], Jyothi A P[1], Chandana B S[1], Pranathi M[1]

[1]M. S. Ramaiah University of Applied Sciences, Bangalore, India

Mithil K Gowda: mithil.k.g.10@gmail.com; GR Dikshith: dikshithraj03@gmail.com
Shaista Tarannum: Shaistatarannum123@gmail.com; Jyothi A P: jyothi.cs.et@msruas.ac.in
Chandana B S: bschandana0225@gmail.com; Pranathi M: pranathimurali1@gmail.com

*Corresponding author: Mithil K Gowda, mithil.k.g.10@gmail.com

Abstract

Cyberattacks increasingly use coordinated, adaptive, and evasive techniques that are difficult to manage with static security tools. Traditional firewalls and intrusion detection systems remain useful for known threats, but they are often limited when attack behavior changes over time. In this study, we propose a HoneyBee-inspired metaheuristic multi-agent cyber defense framework that integrates machine learning, deep learning, reinforcement learning, and adaptive deception within a unified defense architecture. The framework follows a continuous Detect–Mislead–Neutralize–Learn cycle. In this cycle, distributed agents monitor security events, detect suspicious behavior, redirect selected attackers to deception environments, select mitigation actions, and update defense knowledge from observed outcomes. The detection layer combines XGBoost with convolutional neural network and long short-term memory components. The response layer applies a Deep Q-Network to select autonomous mitigation actions. The deception layer uses an application-level honeypot to support attacker engagement and threat intelligence collection. Experimental evaluation using CIC-IDS2017 data and simulated traffic reports an accuracy of 98.24%, a false positive rate of 1.85%, a receiver operating characteristic area under the curve of 0.992, a mean response time of 420 ms, and a deception engagement rate of 92.40%. These results indicate that an integrated multi-agent defense framework can support adaptive threat detection, autonomous response, and closed-loop learning in cyber defense environments.

Keywords: Cyber defense, HoneyBee-inspired metaheuristic, Multi-agent systems, Intrusion detection, Adaptive deception, Reinforcement learning, Honeypot, Threat intelligence

1 Introduction

Cloud computing, Internet of Things (IoT) systems, and distributed digital services are now widely used in government, business, healthcare, education, and critical infrastructure. These technologies improve connectivity and data sharing. However, they also increase the attack surface available to malicious actors. Modern attackers do not rely only on isolated attacks. They can combine reconnaissance, credential abuse, distributed denial-of-service (DDoS), malware, and web-application exploits to bypass static defense mechanisms [3, 10].

Cyber threats have also become more adaptive over the last decade. Attackers can change their behavior, hide in legitimate network activity, and exploit both technical and human weaknesses. Traditional security measures, including firewalls and intrusion detection systems (IDS), are effective for many known threats. However, signature-based and rule-based tools often fail when attack patterns evolve or when new attack types appear [15]. Therefore, current cyber defense requires systems that can detect abnormal behavior, respond quickly, and learn from new evidence.

Artificial intelligence (AI), machine learning (ML), and deep learning (DL) have improved anomaly detection and cyber threat classification. ML methods can identify suspicious traffic patterns from structured network-flow data, while DL methods can learn complex temporal and spatial attack behavior [1, 7, 13]. However, detection alone is not sufficient. A practical defense system must also support autonomous response, attacker engagement, and continuous improvement. Reinforcement learning (RL) has been studied for adaptive response selection, but many RL-based methods remain separated from deception mechanisms and broader defense architectures [8, 14].

Bio-inspired defense models provide another direction for cyber security design. Honeybee colonies can detect threats, share information, and coordinate defensive behavior without relying on a single central controller. This collective behavior is relevant to cyber defense because modern networks require distributed monitoring, intelligence sharing, and coordinated response. The DIAMoND method has shown that a bee-inspired model can support distributed network intrusion early warning [4]. However, a stronger framework is still needed to integrate intelligent detection, adaptive deception, autonomous response, and continuous learning within one operational cycle.

In this study, we propose a HoneyBee-inspired metaheuristic multi-agent cyber defense framework for adaptive deception and autonomous response. The framework uses a Detect–Mislead–Neutralize–Learn cycle. Detection agents identify suspicious behavior, deception agents redirect selected attacks to controlled honeypot environments, response agents select mitigation actions, and learning agents update future decisions using collected threat intelligence. In addition, the framework uses distributed agent communication to reduce dependence on a single point of control.

1.1 Contributions

The main technical contributions of this study are as follows:

1) We develop a HoneyBee-inspired metaheuristic multi-agent cyber defense framework that integrates threat detection, adaptive deception, autonomous response, and continuous learning in a unified architecture.

2) We design a hybrid detection layer that combines XGBoost with convolutional neural network and long short-term memory components to process structured network-flow features and temporal attack patterns.

3) We formulate an adaptive deception layer that redirects suspicious sessions to an isolated application-level honeypot and uses attacker interaction logs to support threat intelligence collection.

4) We define a reinforcement learning-based response layer using a Deep Q-Network for autonomous mitigation action selection under changing threat conditions.

5) We establish a closed-loop Detect–Mislead–Neutralize–Learn workflow in which validated intelligence from detection, deception, and response outcomes is returned to the learning layer for future model and policy updates.

6) We evaluate the proposed framework using CIC-IDS2017 data and simulated traffic scenarios with accuracy, false positive rate, receiver operating characteristic area under the curve, response time, deception engagement, comparative benchmarking, and ablation analysis.

1.2 Organization of the Chapter

The remainder of this chapter is organized as follows. Section 2 reviews related work on intelligent cyber defense, ML-based detection, DL-based detection, RL-based response, multi-agent defense, and deception technologies. Section 3 presents the research problem, research questions, objectives, novelty, threat model, and expected contribution. Section 4 describes the proposed HoneyBee-inspired architecture and methodology. Section 5 reports the experimental setup, testbed configuration, evaluation metrics, performance results, comparative analysis, and ablation study. Section 6 discusses practical implications, enterprise and critical infrastructure applications, and future research directions. Section 7 concludes the chapter.

2 Review of Literature

Cyber defense research has gradually shifted from signature-based security mechanisms to adaptive and intelligence-driven approaches. Early IDS methods relied mainly on predefined rules and attack signatures. These methods can detect known attacks, but they often perform poorly when attackers modify their behavior or when previously unseen attack patterns appear [15]. Therefore, recent studies have explored ML, DL, RL, multi-agent coordination, moving-target defense, and deception-based mechanisms to improve cyber resilience.

2.1 Machine Learning-Based Detection

ML methods were among the earliest intelligent approaches used for cyber threat detection. Decision trees, random forests, support vector machines, and related classifiers can learn suspicious traffic patterns from network-flow features. These methods are useful for structured traffic data because they can process features such as flow duration, protocol type, packet statistics, and flag counts. However, ML-based IDS models can lose effectiveness when network behavior changes over time. In addition, model performance can be affected by high-dimensional features, class imbalance, and evolving attack distributions [10]. Therefore, periodic retraining and careful feature engineering are often required to maintain detection reliability.

2.2 Deep Learning-Based Detection

DL methods have been applied to intrusion detection because they can learn complex representations from network traffic. Convolutional neural networks (CNNs) can extract local spatial patterns from structured traffic representations, while long short-term

memory (LSTM) networks can model temporal dependencies in traffic sequences. These methods are useful for identifying multi-stage attack behavior and complex traffic anomalies. However, DL-based IDS models may require higher computational resources than conventional ML models. In addition, their decision logic is often harder to interpret, which can limit practical use in security operation centers where analysts require clear explanations of alerts [1, 13].

2.3 Reinforcement Learning in Cyber Defense

RL has been studied for autonomous cyber defense because an RL agent can learn response actions through interaction with its environment. Unlike static rule-based response systems, an RL-based response mechanism can adapt its action policy based on reward feedback. Such methods can support traffic filtering, access control, attack containment, and automated incident response [8, 9]. However, many RL-based approaches focus mainly on response optimization. They often do not integrate deception, attacker engagement, and continuous threat intelligence feedback into a single cyber defense cycle [14].

2.4 Multi-Agent Cyber Defense Systems

Multi-agent defense systems are relevant for modern distributed networks because a single centralized defense component may not scale well across large infrastructures. Distributed agents can observe different parts of the network, share intelligence, and coordinate response actions. Bio-inspired and swarm-based models also support decentralized monitoring and collective decision-making. For example, bee-inspired network intrusion early warning has been studied through DIAMoND, which uses distributed cyber defense principles inspired by hive behavior [4]. However, many multi-agent defense methods focus mainly on detection or early warning. They provide limited support for adaptive deception and closed-loop learning.

2.5 Honeypots, Deception Technologies, and Moving-Target Defense

Deception technologies allow defenders to observe attacker behavior in a controlled environment. Honeypots and honeynets can collect threat intelligence, delay attackers, and reduce exposure of critical assets. However, static deception environments may be detected by experienced attackers. Therefore, adaptive deception and moving-target defense have been studied to increase uncertainty for attackers and improve system resilience [5, 6]. These methods are useful for reducing attacker confidence, but they are often implemented separately from intelligent detection and autonomous response systems.

2.6 Need for an Integrated Approach

Existing research has made important progress in detection, autonomous response, deception, and distributed defense. However, these capabilities are often developed as separate components. This separation can limit the ability of a defense system to detect threats, mislead attackers, neutralize attacks, and learn from the resulting evidence within a single

operational workflow. Therefore, an integrated framework is needed to connect intelligent detection, adaptive deception, autonomous response, and continuous learning in a coordinated multi-agent architecture.

Table 1 summarizes the main related studies and identifies the specific gap addressed by the proposed framework.

Table 1: Summary of related work and research gap

No.	Study	Year	Focus	Method	Main finding	Gap relevant to this study
1	Gueriani et al. [2]	2023	Deep RL for IDS in IoT	Survey of deep RL-based intrusion detection	RL can support adaptive detection and response in IoT security	The study mainly reviews RL for IDS and does not provide a unified deception-based multi-agent framework.
2	Korczyński et al. [4]	2016	Bee-inspired distributed cyber defense	DIAMoND bee-inspired distributed early warning	Distributed hive-based defense can support early warning for network intrusion	The approach does not fully integrate adaptive honeypots, autonomous RL response, and continuous learning.
3	Okhravi et al. [5]	2016	Moving-target cyber defense	Uncertainty-based moving-target techniques	Moving-target mechanisms can improve defense by increasing attacker uncertainty	The focus is moving-target defense rather than combined detection, deception, response, and learning.
4	Sun et al. [6]	2023	Self-adaptive moving-target defense	Survey of optimized and adaptive moving-target defense	Adaptive defense can reduce predictability and improve cyber resilience	The survey does not present an implemented HoneyBee-inspired multi-agent response architecture.
5	Bacha et al. [14]	2023	Multi-agent deep RL for IDS	Centralized and decentralized multi-agent deep RL approaches	Multi-agent RL can improve IDS decision-making under distributed settings	The approach focuses on intrusion detection improvement and does not directly integrate adaptive deception.
6	Proposed framework	2025	Integrated cyber defense	XGBoost, CNN-LSTM, DQN response, Kafka-based agent communication, and adaptive honeypot	The framework links detection, deception, autonomous response, and learning through the Detect–Mislead–Neutralize–Learn cycle	This study addresses the integration gap by combining multi-agent coordination with adaptive deception and closed-loop learning.

3 Research Challenges and Objectives

3.1 Research Problem

Although intrusion detection, cyber deception, and autonomous response have improved in recent years, many cybersecurity solutions remain fragmented. Existing systems often focus on one major capability, such as threat detection, automated response, deception, or threat intelligence collection. This separation can slow response actions and reduce adaptability when attacks change over time.

A major limitation of current defense architectures is the absence of a unified process that can detect attacks, mislead attackers, neutralize malicious activity, and learn from the resulting evidence. Detection systems may identify suspicious behavior, but they do not always support adaptive deception or autonomous mitigation. Similarly, deception technologies may collect useful attacker information, but they often remain disconnected from detection model updates and response policy learning. Therefore, a unified multi-agent cyber defense framework is needed to integrate intelligent detection, adaptive deception, autonomous response, and continuous learning.

3.2 Research Questions

This study is guided by the following research questions:

1) Can a HoneyBee-inspired multi-agent architecture improve threat detection and response efficiency compared with conventional cyber defense approaches?

2) How effectively can adaptive deception mechanisms support threat intelligence collection and attacker engagement?

3) Can reinforcement learning support autonomous response selection while reducing containment and mitigation time?

4) Does continuous learning from attacker interactions improve the long-term adaptability of cyber defense systems?

3.3 Research Objectives

The main objective of this study is to develop a HoneyBee-inspired metaheuristic multi-agent cyber defense framework that integrates intelligent detection, adaptive deception, autonomous response, and continuous learning within one coordinated architecture.

The specific objectives are as follows:

1) To develop an intelligent threat detection mechanism using ML and DL techniques.

2) To design an adaptive deception layer that can collect useful threat intelligence from suspicious attacker interactions.

3) To implement an RL-based response mechanism for autonomous attack mitigation.

4) To support collaboration among distributed security agents through shared event notifications and threat intelligence records.

5) To establish a continuous learning process that improves future detection and response performance using evidence collected from detection, deception, and response outcomes.

3.4 Novelty of the Proposed Framework

The proposed framework differs from conventional cyber defense systems because it combines four core capabilities in one multi-agent architecture: intelligent detection, adaptive deception, autonomous response, and continuous learning. Many ML-based IDS solutions support detection, but they are not fully adaptive. RL-based methods can improve response selection, but they often do not include deception-based attacker engagement. Honeypot-based systems can collect valuable attacker information, but they do not always use this information to update future detection and response decisions.

In contrast, the proposed framework follows a continuous Detect–Mislead–Neutralize–Learn cycle. The framework is inspired by HoneyBee colony behavior, where distributed agents share information and coordinate defensive action. The technical novelty lies in linking adaptive deception and autonomous response through shared threat intelligence. In the proposed design, the Kafka-based event bus acts as the agent communication mechanism. Threat intelligence collected by deception agents is treated as a form of agent-to-agent signaling. This information is then used as state feedback for the Deep Q-Network (DQN), which supports future response policy updates without requiring centralized manual orchestration.

3.5 Threat Model

The proposed framework assumes a distributed enterprise network that contains user devices, servers, cloud services, authentication systems, and business applications. The protected assets include sensitive organizational data, application servers, authentication services, network services, and operational systems.

The threat model considers both external and internal attackers. These attackers may perform reconnaissance, brute-force login attempts, DDoS attacks, credential theft, web-application attacks, malware deployment, and privilege-escalation attempts. The attacker is assumed to have partial knowledge of exposed network services. However, the attacker does not have access to the internal decision-making logic of the cyber defense framework.

The defender is assumed to have monitoring capability over network traffic, system logs, login events, and security alerts. Monitoring agents are placed across the infrastructure to collect security events. Detection agents analyze collected events and assign threat-confidence scores. Deception agents redirect selected suspicious sessions to isolated honeypot environments. Response agents apply mitigation actions such as monitoring, rate limiting, redirection, isolation, or blocking. Learning agents use detection results, attacker interaction logs, and response outcomes to update future defense behavior.

The boundary between the major layers is enforced through software-defined networking (SDN) rules and edge-level control policies. Detection operates passively on mirrored traffic and collected logs. Deception operates within an isolated honeypot subnet. Response actions are applied through edge-level controls, including blocking and redirection. This separation helps prevent suspicious sessions from reaching critical assets while preserving the ability to observe attacker behavior in a controlled environment.

3.6 Expected Contribution

The proposed study is expected to contribute an integrated cyber defense framework that supports resilience, self-improvement, and adaptive response. The framework embeds ML, DL, RL, and deception technologies within a continuous Detect–Mislead–Neutralize–Learn cycle. In addition, the architecture uses attacker interactions as a source of threat intelligence. This intelligence supports future detection and response decisions, which can reduce manual effort and improve adaptability against changing attack behavior.

4 HoneyBee-Inspired Architecture and Methodology

4.1 Design Philosophy

The proposed framework is inspired by the collective defense behavior of HoneyBee colonies. In a natural colony, different bees monitor the environment, detect threats, share information, and coordinate defensive actions without complete dependence on a single central controller. We adapt this principle to cyber defense by assigning specialized tasks to distributed software agents. These agents monitor network events, identify suspicious activity, redirect selected attackers to deception environments, select mitigation actions, and update defense knowledge from observed outcomes.

Figure 1 presents the overall multi-agent cybersecurity framework. The framework contains four major functional layers: detection, response, deception, and learning. The detection layer uses the hybrid XGBoost and CNN-LSTM model to identify suspicious network behavior. The response layer uses multi-agent coordination and reinforcement learning to select suitable actions. The deception layer uses adaptive honeypots to collect attacker interaction data. The learning layer stores threat intelligence and returns feedback to update the detection and response models.

Table 2 shows how HoneyBee colony behavior is mapped to cyber defense functions in the proposed framework. This mapping converts the biological analogy into a functional cyber defense design.

Table 2: Mapping of HoneyBee behavior to cyber defense functions

HoneyBee behavior	**Cyber defense function**
Scout bees	Threat monitoring agents that observe network traffic, login events, and system logs.
Guard bees	Threat detection agents that evaluate suspicious behavior and generate threat-confidence scores.
Waggle dance communication	Agent-to-agent intelligence sharing through event notifications and shared threat records.
Colony defense	Coordinated response actions, including monitoring, rate limiting, redirection, isolation, and blocking.
Collective learning	Threat intelligence repository and model update process based on detection, deception, and response outcomes.

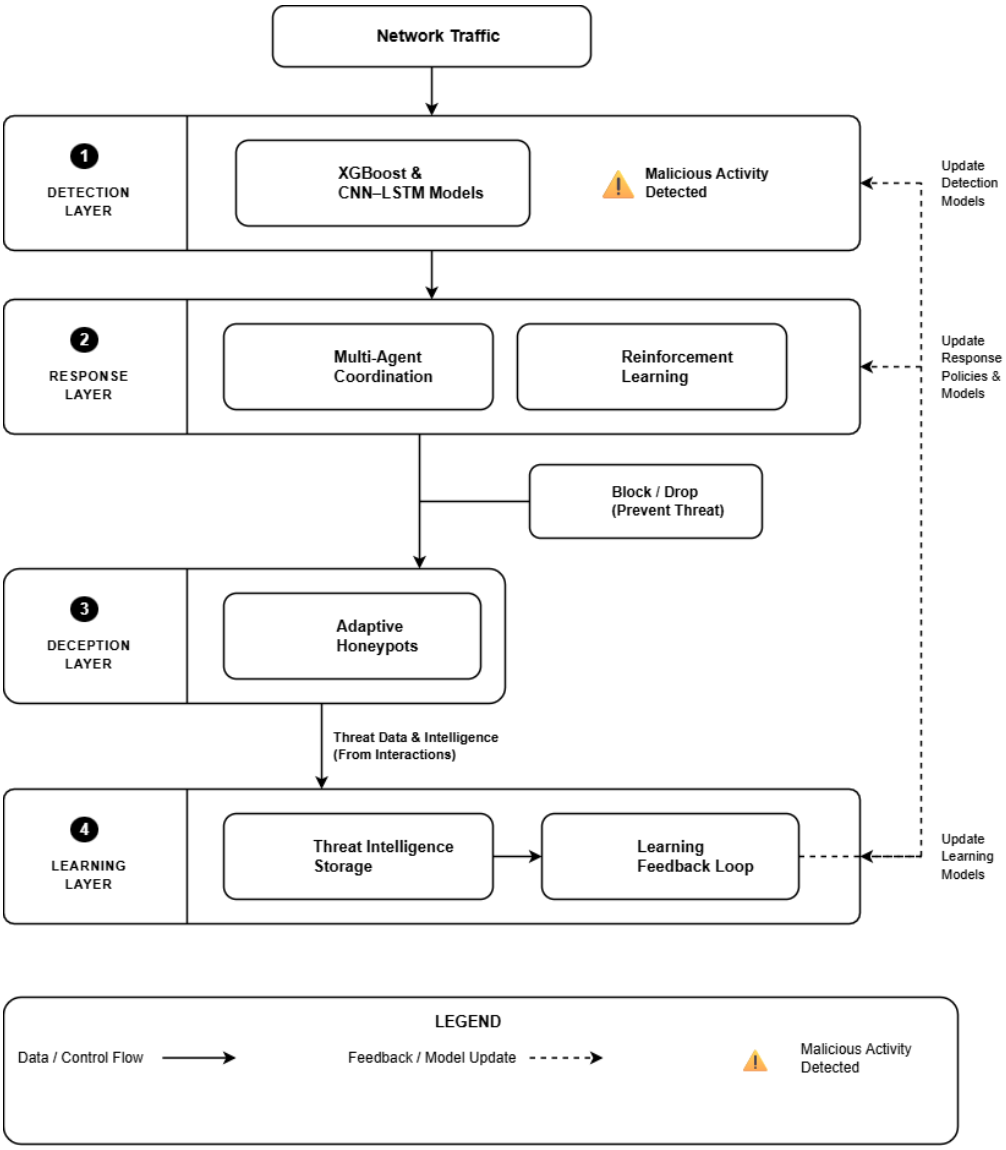

Figure 1: Multi-agent cybersecurity framework

4.2 Formal System Model

The proposed multi-agent framework is modeled as a decentralized and partially observable decision process. The environment contains network traffic, system logs, authentication events, honeypot interaction logs, and response outcomes. Let $\mathcal{S}$ denote the environment state space, where each state $s_t \in \mathcal{S}$ at time t summarizes the current threat context. The state can include the hybrid threat score, attack category, confidence level, honeypot engagement status, and recent response history.

Each agent receives an observation $o_t \in \mathcal{O}$ rather than full access to the complete system state. Monitoring agents produce feature vectors from traffic and logs. Detection agents compute a unified threat score. Response agents select mitigation actions from the action space $\mathcal{A}$. Deception agents collect attacker interaction records and publish threat intelligence to the shared communication layer.

The hybrid threat score is computed by combining the XGBoost output and the CNN-LSTM output as shown in Eq. (1):

$$T_t = \lambda P_{\text{XGB}}(x_t) + (1 - \lambda) P_{\text{CNN-LSTM}}(x_t), \tag{1}$$

where T_t is the unified threat score at time t, x_t is the input feature vector, $P_{\text{XGB}}(x_t)$ is the threat probability predicted by the XGBoost model, $P_{\text{CNN-LSTM}}(x_t)$ is the threat probability predicted by the CNN-LSTM model, and $\lambda \in [0, 1]$ is the fusion weight. A suspicious event is forwarded to the response layer when T_t is greater than a predefined threshold τ, as shown in Eq. (2):

$$d_t = \begin{cases} 1, & T_t \geq \tau, \\ 0, & T_t < \tau, \end{cases} \tag{2}$$

where $d_t = 1$ indicates that the event is treated as suspicious and $d_t = 0$ indicates that the event is treated as benign.

4.3 Architectural Overview

The framework consists of four specialized agent groups: monitoring agents, detection agents, response agents, and learning agents. Monitoring agents collect raw network traffic, login activity, and system logs. Detection agents process collected events and generate threat-confidence scores. Response agents select mitigation actions such as monitoring, rate limiting, redirection, isolation, and blocking. Learning agents analyze historical attack records, honeypot interaction logs, and response outcomes to improve future detection and response behavior.

Agent communication is handled through shared event notifications and threat intelligence records. In the proposed implementation, Apache Kafka is used as the publish-subscribe communication layer. Detection outputs are sent to the response layer, while attacker interaction records from the deception layer are returned to the learning layer. This communication structure supports a closed-loop defense cycle without requiring all actions to be managed by a single central component.

4.4 Detection Layer

The detection layer uses a hybrid XGBoost and CNN-LSTM architecture. Network traffic and system logs are collected through the communication layer and then processed

before model inference. Preprocessing includes duplicate removal, missing value handling, feature normalization, and categorical feature transformation. Extracted features include packet statistics, flow duration, protocol type, flag counts, login frequency, session behavior, and temporal traffic patterns.

XGBoost is used as the first-stage classifier for structured network-flow features. This component supports fast classification of a large proportion of benign and suspicious traffic. The CNN-LSTM component analyzes spatial and temporal dependencies to identify complex attack patterns. The CNN component extracts local feature patterns, while the LSTM component captures session-level temporal dependencies. The outputs of both models are fused into the unified threat score defined in Eq. (1). Table 3 summarizes the detection model configuration.

Table 3: Detection model configuration

Model component	**Hyperparameters or configuration**
XGBoost stage	Learning rate: 0.1; maximum depth: 6; estimators: 100.
CNN spatial component	One-dimensional convolutional layers; 64 filters; kernel size: 3.
LSTM temporal component	50 units; early stopping with patience of 10 epochs.
Optimizer	Adam optimizer.
Class balancing	Synthetic Minority Over-sampling Technique (SMOTE).

4.5 Deception Layer

The deception layer uses a custom application-level deception engine rather than a generic static honeypot. When the detection layer identifies suspicious behavior, SDN rules redirect the suspicious session to an isolated Flask-based honeypot environment on Port 5001. The honeypot is designed to imitate a realistic application-level environment while preventing direct access to sensitive assets.

To preserve believability, the honeypot queries the real intelligence database in read-only mode and applies a deterministic column-wise derangement algorithm. This process produces structurally realistic but synthetically scrambled data. Therefore, attackers can interact with realistic-looking records without receiving actual sensitive values. All attacker interactions, search queries, submitted payloads, and commands are recorded in a local SQLite database named `honeypot_access`. The recorded information is then streamed through Apache Kafka to the learning layer for policy and model updates.

4.6 Response Layer

The response layer uses a DQN agent to select mitigation actions based on the current threat context. The autonomous response problem is modeled as a Markov decision process (MDP), as shown in Eq. (3):

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle, \tag{3}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, P is the transition function, R is the reward function, and γ is the discount factor. The state space includes the threat score,

attack category, historical behavior, confidence level, and honeypot engagement status. The action space includes monitoring, rate limiting, honeypot redirection, IP blocking, and escalation for manual inspection.

The reward function encourages rapid containment and useful intelligence collection while reducing false positives and unnecessary service disruption. Positive rewards are assigned for successful containment and successful threat intelligence collection. Penalties are assigned for delayed response, incorrect blocking of legitimate traffic, and excessive resource consumption. The general reward function is shown in Eq. (4):

$$R(s_t, a_t) = r_{\text{contain}} + r_{\text{intel}} - c_{\text{fp}} - c_{\text{delay}} - c_{\text{resource}}, \tag{4}$$

where $R(s_t, a_t)$ is the reward for action a_t in state s_t, r_{contain} is the reward for successful threat containment, r_{intel} is the reward for useful intelligence collection, c_{fp} is the penalty for false positive action, c_{delay} is the penalty for delayed mitigation, and c_{resource} is the penalty for resource overhead.

The DQN policy updates its Q-values using the Bellman update rule shown in Eq. (5):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[R(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right], \tag{5}$$

where $Q(s_t, a_t)$ is the expected future utility of selecting action a_t in state s_t, α is the learning rate, $R(s_t, a_t)$ is the immediate reward, γ is the discount factor, s_{t+1} is the next state, and $\max_{a'} Q(s_{t+1}, a')$ is the maximum expected future utility over possible next actions. The learning episode terminates when the threat is contained, the event is classified as benign, or the predefined observation window is reached. Table 4 summarizes the DQN configuration.

Table 4: Reinforcement learning configuration for the response layer

Parameter	**Value or description**
State space	Threat score, attack category, confidence level, historical behavior, and honeypot engagement status.
Action space	Monitor, rate limit, redirect to honeypot, block, and escalate for manual inspection.
Reward function	Positive reward for containment and intelligence collection; penalty for false positives, delay, and resource cost.
Policy type	Deep Q-Network.
Update rule	Bellman Q-value update.
Safety validation	Response actions are checked against predefined security policies before execution.

4.7 Learning Layer

The learning layer supports continuous adaptation of the framework. It receives information from detection results, honeypot engagement logs, and response outcomes. This information is used to update detection models and response policies. The feedback process helps the system improve its future decisions when attack behavior changes. Each validated security incident is treated as an evidence source for model refinement and policy adjustment.

4.8 Multi-Agent Coordination

The framework coordinates four specialized agent types through an Apache Kafka publish-subscribe protocol. Each agent has a defined input, processing role, and output. Table 5 summarizes the major agent roles.

Table 5: Roles of agents in the proposed framework

Agent type	**Input**	**Main function**	**Output**
Monitoring agent	Raw packets, login events, and system logs	Collects and normalizes security events	Feature vectors and event records.
Detection agent	Feature vectors and event records	Applies the XGBoost and CNN-LSTM pipeline	Unified threat-confidence score.
Deception agent	Redirected suspicious sessions	Records attacker payloads, commands, queries, and dwell time	Honeypot intelligence records.
Response agent	Threat score and system state	Executes the DQN policy and applies mitigation commands	Monitor, rate limit, redirect, block, or escalate action.
Learning agent	Detection outputs, response outcomes, and honeypot logs	Updates detection models and response policies	Updated model parameters and policy feedback.

Fallback handling is included to reduce the risk of operational failure. If the central learning process is delayed, response agents can temporarily apply localized rule-based isolation. This mechanism helps maintain defensive continuity until the learning layer becomes available again.

4.9 Operational Workflow

The operational workflow begins with continuous monitoring of network traffic and system events. Monitoring agents convert events into normalized feature vectors. Detection agents compute the threat score. If the score exceeds the threshold τ, the response layer selects an action using the DQN policy. If the selected action is redirection, the suspicious session is sent to the deception layer. Otherwise, the selected mitigation action is executed directly. The learning layer then updates future detection and response behavior using the observed outcome. Figure 2 illustrates the Detect–Mislead–Neutralize–Learn workflow, and Algorithm 1 provides the corresponding procedural description.

4.10 Advantages of the Architecture

The proposed architecture offers several advantages over isolated cyber defense mechanisms. First, the distributed agent design supports scalability and reduces dependence on a single defense component. Second, the adaptive deception layer allows the system to collect threat intelligence from attacker interactions. Third, the DQN-based response layer supports autonomous action selection under changing attack conditions. Finally,

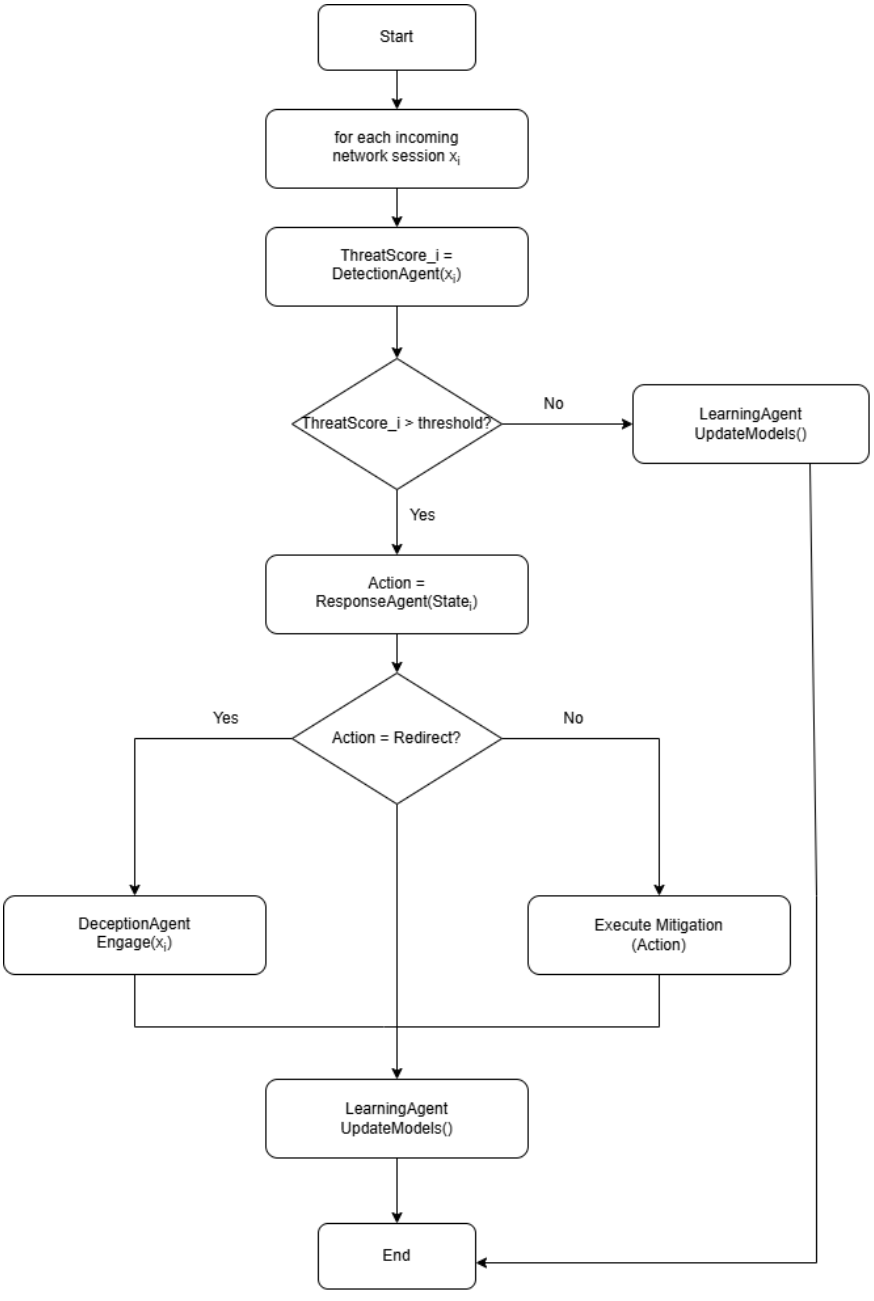

Figure 2: Detect–Mislead–Neutralize–Learn workflow

Algorithm 1 Detect–Mislead–Neutralize–Learn workflow

Require: Incoming network event stream $\mathcal{E}$, threat threshold τ, trained detection models, DQN response policy

Ensure: Mitigation action and updated defense knowledge

for each incoming event $e_t \in \mathcal{E}$ **do**
    Monitoring agents extract the feature vector x_t from e_t.
    Detection agents compute $P_{\text{XGB}}(x_t)$ and $P_{\text{CNN-LSTM}}(x_t)$.
    Compute the unified threat score T_t using Eq. (1).
    if $T_t \geq \tau$ **then**
        Construct the current state s_t from the threat score, attack category, confidence level, and engagement status.
        Response agents select action a_t using the DQN policy.
        if a_t is honeypot redirection **then**
            Deception agents redirect the session to the isolated honeypot.
            Record attacker interaction data, commands, payloads, and dwell time.
        else
            Execute the selected mitigation action.
        end if
        Compute the reward $R(s_t, a_t)$ using the response outcome.
        Update the DQN policy using Eq. (5).
        Learning agents update detection and response knowledge from validated records.
    else
        Mark the event as benign and continue monitoring.
    end if
end for

the learning layer closes the feedback loop by using detection results, response outcomes, and deception logs to improve future decisions. Therefore, the framework supports a proactive cyber defense process rather than a purely reactive detection mechanism.

5 Results and Discussion

5.1 Experimental Setup

We evaluated the proposed framework using the CIC-IDS2017 dataset and a controlled simulated traffic environment. The CIC-IDS2017 dataset was developed by the Canadian Institute for Cybersecurity at the University of New Brunswick. It contains approximately 2.8 million network-flow records with 78 traffic features. The dataset includes benign traffic and several attack categories, including DoS, DDoS, Port Scan, Brute Force, Web Attacks, Botnet, Heartbleed, and Infiltration.

To improve attack diversity and emulate operational conditions, the benchmark data were supplemented with synthetic traffic generated within the simulation environment. The synthetic traffic included normal user behavior, authentication events, reconnaissance activity, login attempts, and multi-stage attack scenarios. Feature extraction and harmonization were performed to create a unified dataset for training and evaluation. The preprocessing stage included duplicate removal, missing value handling, feature normalization, categorical feature transformation, and class balancing using the Synthetic Minority Over-sampling Technique (SMOTE). The processed data were divided into training, validation, and testing subsets with a 70:15:15 split. Table 6 summarizes the dataset configuration.

Table 6: Dataset details used for experimental evaluation

Feature	**Description**
Dataset name	CIC-IDS2017.
Total records	Approximately 2.8 million network-flow records.
Features extracted	78 network-flow attributes.
Attack categories	DoS, DDoS, Port Scan, Brute Force, Web Attacks, Botnet, Heartbleed, and Infiltration.
Data split	70% training, 15% validation, and 15% testing.
Preprocessing	Duplicate removal, missing value handling, normalization, categorical feature transformation, and SMOTE-based class balancing.
Supplementary traffic	Simulated normal user behavior, authentication events, reconnaissance activity, login attempts, and multi-stage attack scenarios.

5.2 Experimental Testbed

The proposed framework was evaluated in a controlled virtualized simulation environment designed to reproduce enterprise network operations. The testbed included logical hosts representing legitimate users, attacker nodes, defense agents, and isolated deception environments. These components were connected using software-defined network

management policies. Network telemetry, authentication events, and system logs were streamed through Apache Kafka and processed by the hybrid XGBoost and CNN-LSTM detection pipeline.

Python-based attack generation scripts were used to simulate denial-of-service attacks, reconnaissance activity, brute-force authentication attempts, web-application attacks, and infiltration attempts under varying traffic conditions. Suspicious sessions were redirected to a segregated Flask-based deception environment for intelligence collection. Response actions were selected by the DQN-based decision engine. Each attack scenario was executed repeatedly under different traffic loads to evaluate detection accuracy, response latency, deception engagement, and adaptive learning behavior. Table 7 presents the experimental testbed configuration.

Table 7: Experimental testbed configuration

Component	**Configuration**
Data streaming	Apache Kafka message broker and ZooKeeper cluster management.
Model environment	Python virtual environment.
Honeypot platform	Custom application-level Flask deception engine on Port 5001.
Machine-learning libraries	scikit-learn, TensorFlow/Keras, and XGBoost.
Operating system	Windows local environment.
Attack scenarios	Denial-of-service, reconnaissance, brute-force authentication, web-application attacks, and infiltration attempts.
Response mechanism	DQN-based response engine with mitigation through monitoring, rate limiting, redirection, blocking, or escalation.

5.3 Evaluation Metrics

The detection performance was evaluated using confusion-matrix-based classification metrics. Let true positives (TP) denote correctly detected attacks, true negatives (TN) denote correctly detected benign traffic, false positives (FP) denote benign traffic incorrectly classified as malicious, and false negatives (FN) denote attacks missed by the system. The main detection metrics were computed using Eqs. (6)–(10).

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \tag{6}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{7}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{8}$$

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{9}$$

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}. \tag{10}$$

Accuracy measures the overall proportion of correctly classified traffic records. Precision measures the proportion of alerts that correspond to actual attacks. Recall measures the proportion of actual attacks detected by the model. The F1-score balances precision and recall and is useful when traffic classes are imbalanced. The false positive rate (FPR) measures the proportion of benign traffic incorrectly flagged as malicious. In addition, receiver operating characteristic area under the curve (ROC-AUC), mean time to respond, and deception engagement were used to assess detection quality, operational response speed, and attacker interaction effectiveness.

5.4 Main Performance Results

Table 8 reports the main performance results of the proposed framework over 10 independent runs. The framework achieved a mean accuracy of 98.24% with a 95% confidence interval of ±0.15%. The precision, recall, and F1-score values were also above 98%, which indicates balanced detection behavior across attack and benign traffic classes. The ROC-AUC value of 0.992 shows strong class separation. The FPR was 1.85%, which indicates that the framework maintained a low false alarm rate. The reported mean time to respond was 420 ms, and the deception engagement rate was 92.40%.

Table 8: Main performance results of the proposed framework

Metric	**Mean performance over 10 runs**	**95% confidence interval**
Accuracy	98.24%	±0.15%
Precision	98.51%	±0.22%
Recall	98.10%	±0.18%
F1-score	98.30%	±0.19%
ROC-AUC	0.992	±0.003
False positive rate	1.85%	±0.20%
Mean time to respond	420 ms	±45 ms
Deception engagement	92.40%	±1.50%

These results indicate that the hybrid detection approach can identify malicious activity while limiting false alarms. The XGBoost component supports rapid classification of structured network-flow features, while the CNN-LSTM component supports analysis of temporal and spatial attack behavior. This combination improves the ability of the system to process both known and complex attack patterns.

5.5 Detection Performance Analysis

The proposed detection layer combines fast feature-based classification and temporal sequence analysis. XGBoost processes structured features such as protocol type, flow duration, packet statistics, and flag counts. The CNN-LSTM model analyzes temporal behavior in traffic sessions. The fusion of these model outputs produces the unified threat score used by the response layer.

The high precision value indicates that most alerts generated by the system corresponded to actual malicious activity. The high recall value indicates that the system

detected most attack instances in the evaluation set. The low FPR is important for operational deployment because excessive false alarms can increase analyst workload and may cause unnecessary service disruption. Therefore, the combined detection results support the use of the hybrid detection design in the proposed multi-agent framework.

5.6 Deception and Intelligence Collection

The deception layer was used to collect information from suspicious sessions redirected to the isolated honeypot environment. This layer allowed attacker behavior to be observed in a controlled setting. The recorded information included access attempts, commands, search queries, submitted payloads, and dwell time. These records were treated as threat intelligence and streamed to the learning layer for later model and policy updates.

The reported deception engagement rate of 92.40% suggests that the application-level deception environment was effective in sustaining attacker interaction during the evaluation. This result is important because deception does not only delay or divert attackers. It also generates evidence that can improve future detection and response decisions. However, all deception-based deployment should be configured with strict isolation and privacy safeguards to prevent exposure of real assets or sensitive data.

5.7 Autonomous Response Evaluation

The response layer used the DQN policy to choose mitigation actions according to the current threat context. The available actions included monitoring, rate limiting, honeypot redirection, IP blocking, and escalation for manual inspection. The reward function encouraged successful containment and threat intelligence collection, while penalizing false positives, delay, and excessive resource use.

The reported mean response time of 420 ms indicates that the framework can select mitigation actions quickly in the simulated environment. In addition, the DQN mechanism allows response behavior to adapt based on observed outcomes. This capability is important because static response rules may not remain effective when attackers change their tactics. For safety, high-impact actions such as permanent blocking or isolation should be validated through predefined security policies and may require human approval in real deployments.

5.8 Multi-Agent Coordination Analysis

The multi-agent design allowed monitoring, detection, deception, response, and learning tasks to operate as coordinated but specialized functions. Kafka-based communication supported asynchronous exchange of event records and threat intelligence. This design reduced direct dependence on a single centralized decision process. It also allowed information collected from deception and response outcomes to be returned to the learning layer.

The coordination mechanism is important for distributed cyber defense because large enterprise networks may contain many monitoring points and heterogeneous services. By assigning specific roles to agents, the framework supports modular deployment and easier extension. In addition, fallback handling allows response agents to apply localized rule-based isolation when the learning process is delayed.

5.9 Comparative Performance Analysis

Table 9 compares the proposed framework with selected baseline variants under the reported evaluation setting. The proposed framework achieved higher accuracy and lower FPR than the standalone deep IDS baseline and the RL-IDS baseline. It also includes adaptive deception, which is not present in the two baseline variants.

Table 9: Comparative performance analysis with related cyber defense framework variants

Framework variant	Algorithm or approach	Accuracy	FPR	Adaptive deception
Proposed framework	Hybrid XGBoost and CNN-LSTM with RL response and honeypot	98.24%	1.85%	Yes
Deep IDS baseline	Standalone CNN-LSTM	93.20%	5.20%	No
RL-IDS baseline	Static DQN response	95.00%	4.00%	No

The comparison indicates that integrating detection, deception, response, and learning improves overall defense performance compared with isolated baseline variants. The standalone CNN-LSTM model supports detection but does not include autonomous response or deception. The RL-IDS baseline supports response selection but does not include adaptive attacker engagement through a honeypot. Therefore, the proposed architecture offers broader operational coverage than detection-only or response-only variants.

Table 10 presents the comparative benchmarking against prior research and baseline systems reported in the chapter. The proposed system shows higher detection accuracy, lower FPR, and lower mean time to respond than the listed alternatives. However, these results should be interpreted with attention to differences in implementation, datasets, and evaluation protocols across prior work.

Table 10: Comparative benchmarking against prior research and baseline systems

System model	Architecture type	Detection accuracy	FPR	Mean time to respond
Proposed system	Multi-agent RL with adaptive deception	98.24%	1.85%	420 ms
DIAMoND [4]	Static swarm-based non-learning defense	90.00%	7.50%	Approximately 1500 ms
RL-IDS baseline [2]	Single-agent reinforcement learning	95.00%	4.00%	Approximately 1000 ms
Deep IDS baseline [13]	Deep learning-based detection only	93.20%	5.20%	Not applicable

5.10 Ablation Study

The ablation study assessed the contribution of major framework components by removing selected capabilities from the full system. Table 11 shows that the full framework

achieved the highest accuracy, the lowest FPR, and the highest deception engagement. The detection-only variant produced lower accuracy and a higher FPR because it lacked both autonomous response and deception feedback. The no-deception variant reduced attacker engagement to zero and also increased the FPR. The no-RL variant showed lower performance than the full framework because static response rules could not adapt to changing threat conditions. The single-agent variant showed the weakest performance because it lacked coordinated multi-agent intelligence sharing.

Table 11: Ablation study of major framework components

Framework variant	**Accuracy**	**False positive rate**	**Deception engagement**
Full proposed framework	98.24%	1.85%	92.40%
Detection-only variant without RL and honeypot	94.10%	4.50%	Not applicable
No-deception variant without honeypot	95.35%	3.20%	0.00%
No-RL variant with static response rules	96.12%	2.80%	85.00%
Single-agent variant without coordination	91.80%	5.90%	60.10%

The ablation results support the importance of integrating all major components. The adaptive honeypot contributes attacker interaction data that supports learning. The RL response layer improves action selection compared with static rules. The multi-agent communication mechanism supports coordinated defense across monitoring, detection, deception, response, and learning functions.

5.11 Key Findings

The experimental results show four main findings. First, the hybrid XGBoost and CNN-LSTM detection layer achieved high classification performance with a low false positive rate. Second, the application-level deception layer supported attacker engagement and produced threat intelligence for learning. Third, the DQN-based response layer reduced response time and supported adaptive mitigation decisions. Fourth, the ablation results show that detection, deception, RL-based response, and multi-agent coordination each contribute to the overall performance of the proposed framework.

Overall, the results indicate that a HoneyBee-inspired metaheuristic multi-agent architecture can strengthen cyber defense by linking detection, deception, autonomous response, and continuous learning. This integrated design is more suitable for adaptive cyber threats than isolated systems that only detect, only deceive, or only respond.

6 Implications and Future Applications

6.1 Practical Implications

Modern cyber threats are becoming more coordinated, adaptive, and difficult to detect through static defense mechanisms. Therefore, organizations need security systems that can do more than identify suspicious activity. The proposed framework supports this need by combining intelligent detection, adaptive deception, autonomous response, and continuous learning in one operational architecture.

The framework can help security teams respond to threats faster because suspicious behavior is processed through distributed agents and mitigation actions are selected by the DQN response layer. In addition, the deception layer can support threat intelligence collection by redirecting selected suspicious sessions to an isolated honeypot environment. This process allows defenders to observe attacker behavior without exposing critical operational assets. The collected intelligence can also support later investigation, detection model updates, and response policy refinement.

The multi-agent design can reduce dependence on a single central decision point. Monitoring, detection, deception, response, and learning functions are distributed across specialized agents. This design can improve resilience because individual functions can operate with defined roles and fallback behavior. For example, response agents can apply localized rule-based isolation when the learning process is delayed.

6.2 Deployment and Governance Considerations

Although autonomous response can improve response speed, it also introduces operational risks. False positive alerts may lead to unnecessary blocking, redirection, or service disruption. Therefore, high-impact actions such as permanent blocking, system isolation, or escalation affecting legitimate users should be controlled through predefined safety policies. In practical deployment, such actions may require human-in-the-loop approval.

The use of deception technologies also requires careful governance. Honeypot environments must be isolated from real assets, and any attacker interaction logs must be stored securely. In addition, organizations should ensure that deception-based monitoring follows applicable privacy, legal, and compliance requirements. These safeguards are important because the framework collects interaction records, payloads, commands, and behavioral evidence from suspicious sessions.

6.3 Enterprise and Critical Infrastructure Applications

The proposed architecture can be applied in several security environments, including enterprise networks, cloud platforms, smart-city systems, educational institutions, and critical infrastructure networks. Organizations with distributed network assets may benefit from the multi-agent structure because different agents can monitor different network segments and exchange threat intelligence through a shared communication layer.

In enterprise environments, the framework can support security operations by combining real-time detection with automated mitigation and attacker engagement. In cloud environments, the approach can help monitor dynamic workloads and redirect suspicious traffic to controlled deception zones. In critical infrastructure settings, the framework can support early detection, controlled response, and improved situational awareness.

However, deployment in high-risk environments should include strong safety validation, strict access control, and human review for high-impact response actions.

6.4 Future Research Directions

Future research can extend the proposed framework in several directions. First, large language models and advanced threat intelligence systems may be integrated to improve the interpretation of attacker behavior and security event context. Second, privacy-preserving collaborative learning can be explored so that multiple organizations can improve detection capability without directly sharing sensitive data. Third, the framework should be tested in larger real-world environments to assess scalability, robustness, and long-term adaptability under operational constraints.

Future work should also examine adversarial evasion, noisy traffic, concept drift, agent communication delays, and honeypot fingerprinting. These evaluations can clarify how the framework performs when attackers deliberately attempt to avoid detection or identify deception environments. In addition, future deployments should evaluate computational overhead, resource cost, response safety, and analyst workload reduction.

7 Conclusion

This study presented a HoneyBee-inspired metaheuristic multi-agent cyber defense framework for adaptive deception and autonomous response. The framework was designed to address the fragmentation of existing cyber defense systems, where detection, deception, response, and learning are often treated as separate functions. We integrated these functions into a unified Detect–Mislead–Neutralize–Learn cycle supported by distributed agents.

The proposed framework combines an XGBoost and CNN-LSTM detection layer, an application-level adaptive honeypot, a DQN-based autonomous response layer, and a learning layer that uses detection outputs, honeypot interaction logs, and response outcomes for continuous improvement. The HoneyBee-inspired design supports distributed monitoring, agent-to-agent threat intelligence sharing, coordinated response, and collective learning. This design reduces dependence on a single centralized defense component and supports adaptive behavior under changing threat conditions.

The experimental results showed that the proposed framework achieved strong detection and response performance in the evaluated environment. The framework reported an accuracy of 98.24%, a false positive rate of 1.85%, an ROC-AUC value of 0.992, a mean response time of 420 ms, and a deception engagement rate of 92.40%. The comparative and ablation results further indicated that each major component contributed to the overall performance. In particular, the adaptive deception layer supported attacker engagement, the DQN response layer improved mitigation decision-making, and the multi-agent communication mechanism supported coordinated defense.

Overall, the findings indicate that a HoneyBee-inspired multi-agent architecture can provide a useful foundation for adaptive cyber defense. By connecting intelligent detection, attacker deception, autonomous response, and continuous learning, the framework can help transform security events into actionable intelligence. Future work should validate the framework in larger real-world deployments, examine robustness against adversarial evasion and concept drift, and assess deployment safety under strict governance and human-in-the-loop response policies.

References

[1] Zhang, Z., Al Hamadi, H., Damiani, E., Yeun, C.Y., Taher, F.: Explainable artificial intelligence applications in cyber security: State-of-the-art in research. IEEE Access 10, 93104–93139 (2022). https://doi.org/10.1109/ACCESS.2022.3204051

[2] Gueriani, A., Kheddar, H., Mazari, A.C.: Deep reinforcement learning for intrusion detection in IoT: A survey. In: 2023 2nd International Conference on Electronics, Energy and Measurement (IC2EM), pp. 1–6 (2023). https://doi.org/10.1109/IC2EM59347.2023.10419560

[3] Ismail, M.I., Mohmand, H., Khan, A.A., Ullah, U., Zakarya, M., Ahmed, A., Raza, M., Rahman, I.U., Haleem, M.: A machine learning-based classification and prediction technique for DDoS attacks. IEEE Access 10, 21443–21454 (2022). https://doi.org/10.1109/ACCESS.2022.3152577

[4] Korczyński, M., Hamieh, A., Huh, J.H., Holm, H., Rajagopalan, S.R., Fefferman, N.H.: Hive oversight for network intrusion early warning using DIAMoND: A bee-inspired method for fully distributed cyber defense. IEEE Commun. Mag. 54, 60–67 (2016). https://doi.org/10.1109/MCOM.2016.7497768

[5] Okhravi, H., Streilein, W.W., Bauer, K.S.: Moving target techniques: Leveraging uncertainty for cyber defense. Lincoln Lab. J. 22, 100–109 (2016).

[6] Sun, R., Zhu, Y., Fei, J., Chen, X.: A survey on moving target defense: Intelligently affordable, optimized and self-adaptive. Appl. Sci. 13, 5367 (2023). https://doi.org/10.3390/app13095367

[7] Chahal, A.: Harnessing AI and machine learning for intrusion detection in cyber security. Int. J. Sci. Res. 12 (2023). https://doi.org/10.21275/SR231003163943

[8] Saghrouchni, M., Mouël, J., Szanto, K.: Towards an unsupervised reward function for a deep reinforcement learning based intrusion detection system. In: 2024 8th Cyber Security in Networking Conference (CSNet) (2024). https://doi.org/10.1109/CSNet64211.2024.10851732

[9] Odeh, N.: Autonomous reinforcement learning-based intrusion detection for IoT cyber defense. Digital 6, 41 (2026). https://doi.org/10.3390/digital6020041

[10] Nabi, J., Zhou, Y.: Enhancing intrusion detection systems through dimensionality reduction: A comparative study of machine learning techniques for cyber security. Cyber Secur. Appl. 2, 100033 (2024). https://doi.org/10.1016/j.csa.2023.100033

[11] Siregar, V., Hanafi, M.: Design and evaluation of an adaptive intrusion detection framework for IoT edge networks using hybrid machine learning and deep reinforcement learning techniques. Cyber Security and Network Management 1 (2026). https://doi.org/10.66472/cybernet.v1i1.8

[12] Zaccagnino, P., Cirillo, C., Guarino, A., Lettieri, C., Malandrino, D., Zaccagnino, R.: Towards a geometric deep learning-based cyber security: Network system intrusion detection using graph neural networks. In: Proceedings of the 20th International

Conference on Security and Cryptography, pp. 555–562 (2023). https://doi.org/10.5220/0012085700003555

[13] Parampottupadam, S., Moldovann, P.: Cloud-based real-time network intrusion detection using deep learning. In: 2018 International Conference on Cyber Security and Protection of Digital Services, pp. 1–7 (2018). https://doi.org/10.1109/CyberSecPODS.2018.8560674

[14] Bacha, A., Ktata, N., Louati, A.: Improving intrusion detection systems with multi-agent deep reinforcement learning: Enhanced centralized and decentralized approaches. In: Proceedings of the 20th International Conference on Security and Cryptography, pp. 555–562 (2023). https://doi.org/10.5220/0012124600003555

[15] Kim, J., Aminanto, M.E., Tanuwidjaja, H.C.: Intrusion detection systems. In: Network Intrusion Detection Using Deep Learning, SpringerBriefs on Cyber Security Systems and Networks, pp. 15–30. Springer, Singapore (2018). https://doi.org/10.1007/978-981-13-1444-5_2